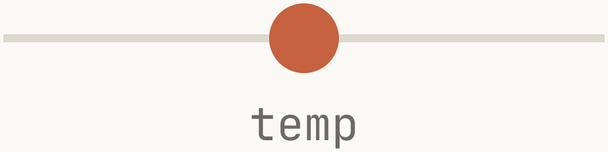
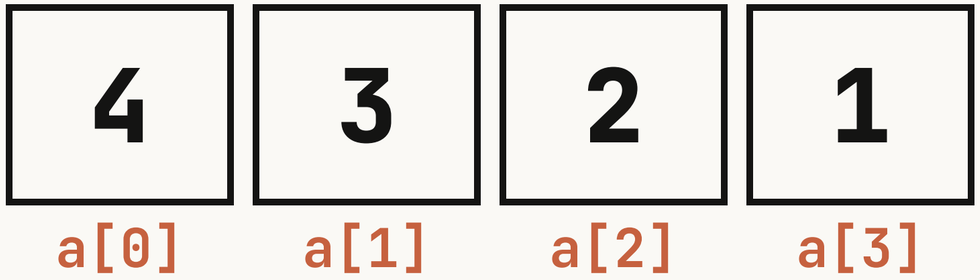
LESSON 21

比邻居  换位置

冒泡排序入门

学完以后，一串乱序的数，程序能一对一对地比，把它们从小到大排好。

一串数，怎样从小到大排好?



人一眼就能看出来该怎么排。程序不行，它每次只能比两个数。

想想同学按身高排队：相邻两人比一比，前面的高就换个位置。
从队头一路比到队尾，再从头来一遍……一直比到没人需要换。
这个办法叫**冒泡排序**：大的数像水里的泡泡，一点点往后冒。

先学一件事：换位置

a 里是 3，b 里是 5。想换成 a 是 5、b 是 3，输入 3 5 时输出 5 和 3。

想象两杯果汁，一杯橙汁、一杯苹果汁，要交换杯子里的东西。
直接把橙汁倒进苹果汁杯？苹果汁就没了。

办法是先拿一个**空杯子**：橙汁倒进空杯，苹果汁倒进橙汁杯，
再把空杯里的橙汁倒进苹果汁杯。

第三个盒子 temp：先存一份

swap.cpp

```
7       int temp = a;
8       a = b;
9       b = temp;
```

输入 3 5

5

3

加一行 temp，第 8 行改成 b = temp。
temp 就是那个空杯子：先把 a 的 3 存进去，
a 再放心拿 b 的 5，最后 b 从 temp 里拿回 3。

读入两个整数，都是 -1000 到 1000，交换后分两行输出 a 和 b。

一轮写成代码：j 走一遍

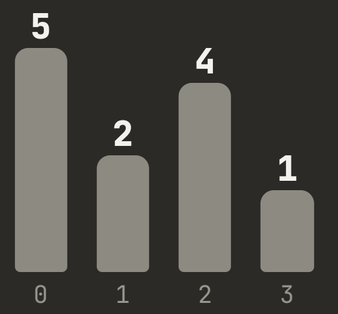
one_pass.cpp

```cpp
    for (int j = 0; j < n - 1; j++) {
        if (a[j] > a[j + 1]) {
            int temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
        }
    }
```

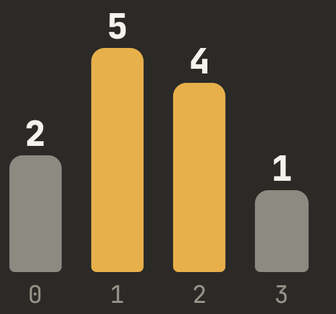
a[j] 和右边的 a[j + 1] 比，前面的大就交换。

j 只走到 n - 2：最后一个数没有右边的邻居。

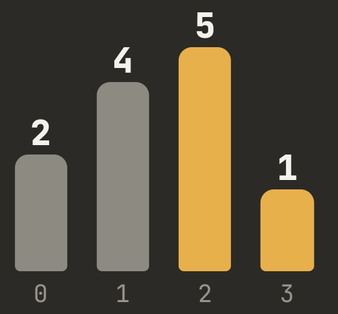
一轮比完，**最大的数一定到了最后**，像泡泡冒上来。



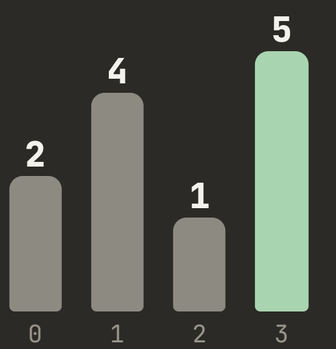
第 1 步 开始：a 是 {5, 2, 4, 1}。挨着的两个比一比。



第 4 步 比 a[1] 和 a[2]：5 比 4 大，要换。



第 6 步 比 a[2] 和 a[3]：5 比 1 大，要换。



第 9 步 比完了。5 已经站好，前面的还没排好。

小测：4 3 2 1 只比一轮，变成什么？

1 1 2 3 4

2 3 2 1 4

3 4 3 2 1

4 1 3 2 4

答对了！4 一路往后换，到了最后；前面的 3 2 1 还没排。

一轮不够，就再来几轮

第 1 轮

最大的到最后

第 2 轮

剩下的里面最大的，到倒数第二

一共 n - 1 轮

前面的都排好了，第一个自然就是最小的

后面排好的不用再比，所以每多一轮，就少比一次。

第 pass 轮（从 0 数）只要比 n - 1 - pass 次。

两层循环：外层数轮，内层比较

sort.cpp

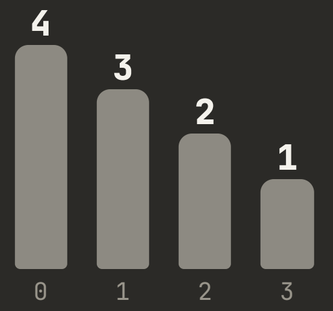
```cpp
    for (int pass = 0; pass < n - 1; pass++) {
        for (int j = 0; j < n - 1 - pass; j++) {
            if (a[j] > a[j + 1]) {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
```

站好的不用再比，所以内层少比 pass 次。

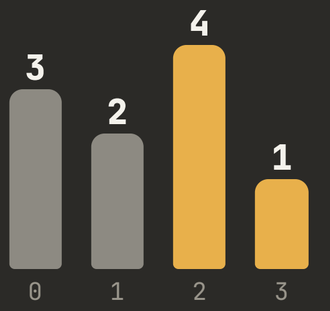
4 个数比 3 轮，一共比较 3 + 2 + 1 = 6 次。

n 是 1 到 20，每个数是 -1000 到 1000。

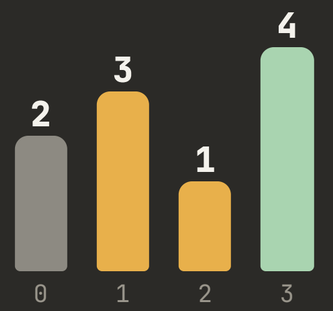
排好后在一行里输出，每个数后面空一格。



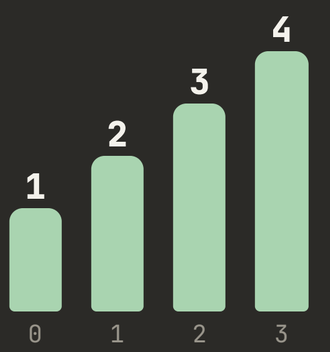
第 1 步 开始：a 是 {4, 3, 2, 1}。挨着的两个比一比。



第 6 步 比 a[2] 和 a[3]：4 比 1 大，要换。



第 10 步 比 a[1] 和 a[2]：3 比 1 大，要换。



第 15 步 全部比完，从小到大排好了。

小测：排好 6 个数，外层要几轮?

1 1 轮

2 5 轮

3 6 轮

4 15 轮

答对了！每轮送一个数到位，5 个到位了，最后一个自然也对了。

动手改一改

从 sort 开始。只改一个符号。

从大到小

读入同上。把这串数从大到小排好，在一行里输出，每个数后面空一格。

输入 3，再输入 3 1 2

```
3 2 1
```

输入 1，再输入 7

```
7
```

Dev-C++：打开 starter/04_descending.cpp

动手改一改

也从 sort 开始。排好以后，正中间的下标是几？

正中间的数

先读入 n，是 1 到 19 的奇数，再读入 n 个整数（-1000 到 1000）。排好以后，只输出正中间的那个数。

输入 5，再输入 4 1 8 3 9

4

输入 1，再输入 7

7

Dev-C++：打开 starter/05_middle.cpp

猜一猜：内层写成 j < n - pass

sort.cpp（只阅读）

```
12          for (int j = 0; j < n - pass; j++) {
```

屏幕上

1 0 2

3 被换到了读入的数据后面，原本不属于输入的 0 被换了进来。数据丢了一个，还多了一个不该有的数。

写 a[j + 1] 时要算清楚：j 最大只能到 n - 2，再减去已经排好的。

读报错：括号没配对

sort.cpp（只阅读）

```
13                  if (a[j] > a[j + 1] {
```

```
13:33: error: expected ')' before '{' token
```

方括号和圆括号挤在一起，最容易漏。

if 的 (还没关，就碰到了 {。在 a[j + 1] 后面补上)。

写完一行，数一数：(和) 一样多，[和] 一样多。

Dev-C++ 显示的文字和位置可能不一样，还是先找行号和 error。

找错：哪一行有问题？

sort.cpp（只阅读）

```
1       if (a[j] > a[j + 1]) {
2           int temp = a[j];
3           a[j] = a[j + 1];
4           a[j + 1] = a[j];
5       }
```

点代码里的一行

这段在交换相邻两个数。

输入 3 个数 3 1 2，排完却是 1 1 2，3 不见了。

点你认为有问题的那一行。

自己挑战

成绩排名

先读入 n（1 到 20），再读入 n 行，每行一个编号（1 到 999）和分数（0 到 100）。按分数从高到低输出编号，每行一个；分数一样时，先输入的排前面。

输入

```
4
101 70
102 95
103 80
104 95
```

输出

```
102
104
103
101
```

先自己写，卡住了再翻下一页。

卡住了？一次看一条提示

提示 1　编号和分数要一起换，用第 19 课的 Record 卡片，一整张一起交换。

提示 2　temp 也可以是一张卡片：Record temp = s[j];

提示 3　比的是 .score，从高到低。一样高时不换，先来的就还在前面。

排好再用

把一串数排好，再用排好的结果做一件有意思的事。主题你自己定。

先做到

用冒泡排序排好一串数，输出排好的样子。

选你喜欢的

跳远前三名、零花钱从少到多、最矮和最高、去掉最高最低分，都可以。

还想更厉害

排一叠 Record 卡片，输出一张排行榜。

没有标准答案。写之前说清想要的结果，写完试试最小和最大的输入。

和同桌换着玩

1

换着运行

运行同桌的程序，试试已经排好的、倒过来的、有一样大的

2

说一个喜欢的

告诉同桌，你最喜欢这个程序哪里

3

提一个想法

问问同桌：排好以后还可以做什么？

听完再改一版。想一想、做出来、试一试、分享、再改，程序就是这样越做越好的。

说得清吗?

Q1 **交换两个数为什么要 temp?**

先放进去的会把原来的数盖掉，temp 先替它存一份。

Q2 **一轮比完，哪个数一定到位了?**

还没排好的里面最大的，它到了这一段的最后。

Q3 **n 个数为什么只要 n - 1 轮?**

每轮送一个到位，n - 1 个到位了，剩下那个自然也对了。

速查卡

```
int temp = a[j]; a[j] = a[j + 1]; a[j + 1] = temp;
```

`pass < n - 1`	外层：一共 n - 1 轮
`j < n - 1 - pass`	内层：每轮少比一次，不越界
`a[j] > a[j + 1]`	从小到大；换成 < 就是从大到小
`temp`	交换时先存一份，别把数盖掉
`a[n / 2]`	排好的奇数个数，正中间那个

这门课学完了，以后还能学

sort()

C++ 自带的排序，比冒泡快得多

vector

大小能自己变的数组

更多算法

二分查找、搜索、动态规划……

从第一行 cout 到冒泡排序，你已经能用 C++ 解决很多真问题了。接着写，接着改，程序会越来越好。

谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

 cpp-for-youth

下一课：sort 排序